

Examen - Implementarea Concurenței în Limbajele de Programare

Optional, Informatica Anul III si CTI Anul IV, Semestrul I, 19 ianuarie 2026

Materiale permise: materialele de curs și cele de laborator, inclusiv fișiere cod cu soluții ale problemelor rezolvate în curs și laborator, în anul universitar curent! Alte fișiere nu sunt acceptate.

În timpul examenului NU AVEȚI VOIE SĂ AVEȚI BROWSERE DESCHISE (cu excepția celor în care rulați cod - onlinegdb.com și onecompiler.com), NU PUTEȚI CONSULTA MATERIALE ONLINE. Nu puteți folosi niciun generator de cod (nici LLM-uri, nici Copilot).

Fiecare student va avea un folder pe care îl va denumi "nume-student-ICLP-2026" în care va avea aceste materiale și în care va rezolva exercițiile. În acest folder, pe parcursul examenului NU pot să apară alte fișiere în afara celor menționate.

La finalul examenului veți face submitia subiectelor rezolvate NUMAI în prezența unui supraveghetor. Vă vom anunța în timpul examenului modul de submitere. Veti trimite fisierele / arhivele cu solutii pe adresele bogdan.macovei@unibuc.ro SI bogdan.macovei.fmi@gmail.com. Subiectul mail-ului va fi ICLP, Nume Prenume, Grupa. Timpul de transmitere este suplimentar timpului efectiv de 2h alocat pentru rezolvarea subiectelor.

1 Subiectul I - Java/Python/Go (maxim 4p)

O resursa este accesata continuu de trei tipuri de procese: scriitor, corector si cititor. Resursa poate fi accesata de:

- maxim N procese cititor (N este fixat la inceput),
- un singur proces scriitor,
- un singur proces corector.

Resursa nu poate fi accesata simultan de mai procese scriitor, de mai multe procese corector sau de mai multe procese de tipuri diferite. Fiecare tip de proces detine resursa un anumit interval de timp, acesta fiind generat aleator. Procesele care acceseaza resursa sunt generate intr-un ciclu infinit, tipul fiecarui process (scriitor, corector sau cititor) fiind generat aleator.

Scrieti un program care implementeaza accesul la resursa, oferind prioritate, in ordine, cititorilor, corectorilor si scriitorilor. Trebuie sa va asigurati ca resursa a fost initial accesata de minim 3 scriitori, pana sa permita celorlalte roluri sa o acceseze.

2 Subiectul al II-lea - Haskell (maxim 4p)

Considerati un model de concurenta inspirat din limbajul Go, care foloseste *task*-uri concurente (echivalentul *goroutines*) si canale prin care task-urile comunica valorile. Implementati un model *Producer-Consumer*, astfel incat *Producer*-ul trimite in canal numerele de la 1 la n si apoi inchide canalul, iar *Consumer*-ul le afiseaza. Comunicarea intre cele doua entitati este posibila daca atat consumatorul, cat si producatorul sunt conectati. Trebuie sa modelati un scenariu in care aveti un *producer* si k (ales initial) *consumeri*. Utilizati urmatoarele tipuri:

```
type Producer = Channel Int -> Int -> IO ()
type Consumer = Channel Int -> IO ()
```

3 Subiectul al III-lea - Erlang (maxim 4p)

Intr-un studio audio, mai multi ingineri de sunet doresc sa foloseasca un singur compresor analogic de mastering. Compresorul poate fi in una dintre stările *free* (asteapta cereri de acces) sau *busy* (este folosit exclusiv de un singur inginer timp de 3 secunde). Fiecare inginer poate fi in una dintre stările *waiting* (a trimis cererea, dar compresorul este ocupat), sau *mastering* (a primit acces la compresor). Accesul se acorda in ordinea cererilor, fara a favoriza niciun inginer de sunet. Implementati acest scenariu folosind modelul actorilor, simuland 5 ingineri care vor sa foloseasca acel compresor analogic.